



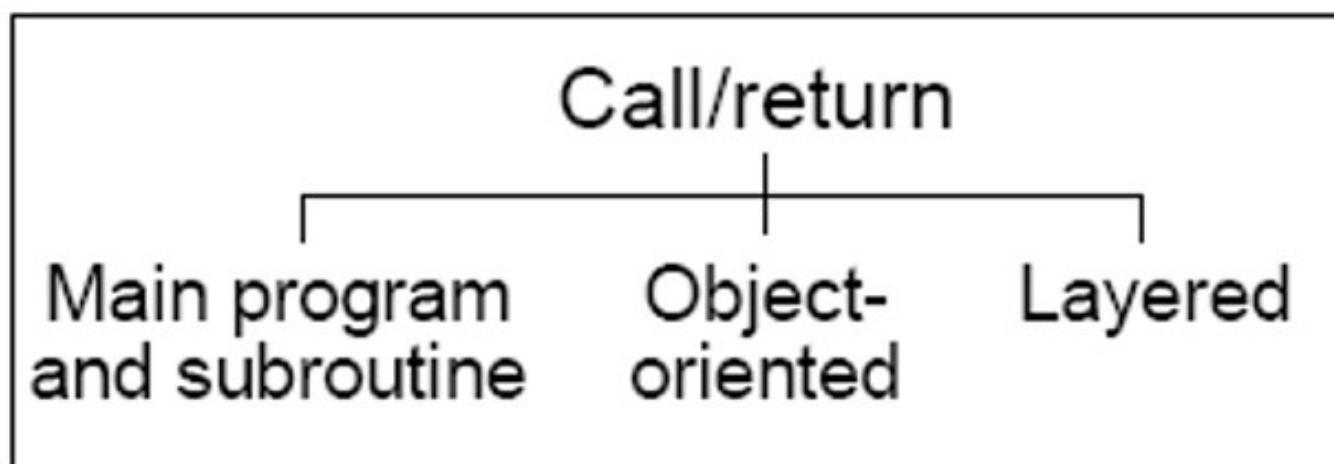
南京理工大学  
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

# 软件体系结构

## 第7章 面向对象

赵庆玲

南京理工大学计算机科学与工程学院



面向对象模式集数据抽象、抽象数据类型、类继承为一体，使软件工程公认的模块化、信息隐藏、抽象、重用性等原则在面向对象模式下得以充分实现。

面向对象模式设计的根本出发点是追求自然的刻划和求解现实世界中的问题，即追求问题空间和软件系统空间结构的一致性。它把系统中的所有资源，如数据、模块等均看作**对象**(一组数据以及可在其上进行的操作所构成的独立单元)，可以通过对象对被求解的问题及其诸要素进行抽象；它将自己的数据结构定义和功能实现封装起来，实现了信息隐藏和数据抽象，体现了模块性，可作为基本构件单元被不同软件系统重用。

## □1、范型(Paradigm)

范型又称为范例、风范或模式(Pattern)。从软件开发角度来看，范型与问题解决技术有关。范型定义了特定的问题和应用的开发过程中将要遵循的步骤，确定用于表示问题和它的解决的那些成分的类型，并利用这些成分表示与问题解决有关的抽象，直接得到问题的结构。因此，范型的选择影响整个软件开发生存期。就是说，它支配了设计方法、编码语言、测试和检验技术的选择。

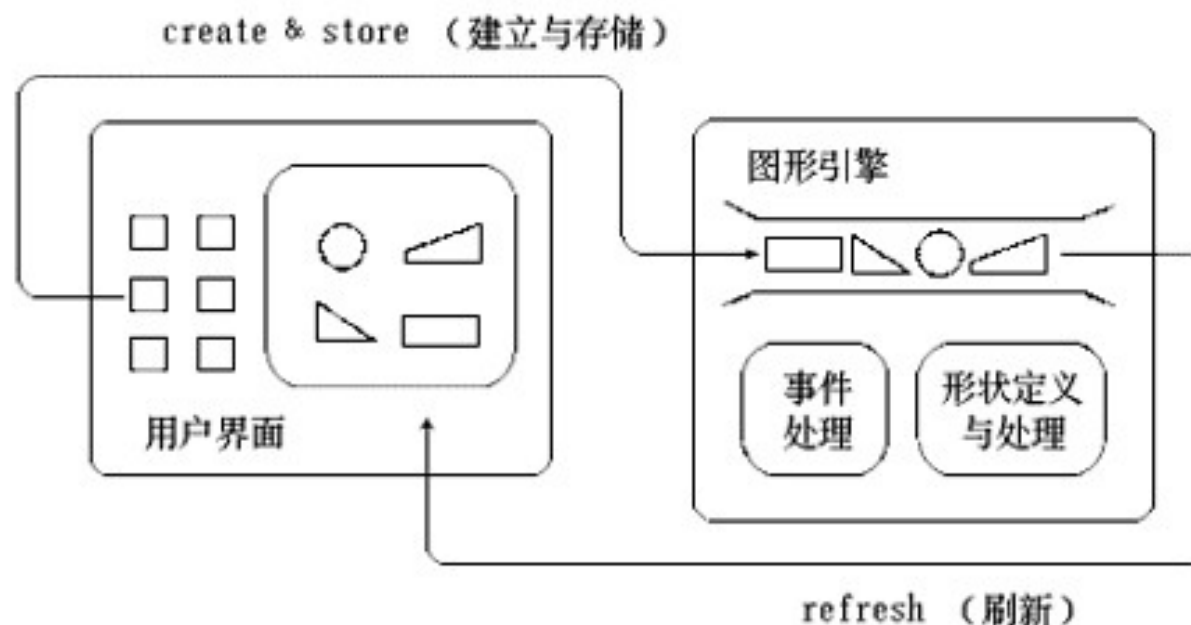
## □2、流行的范型

目前流行多种范型，它们提供了许多方法，可进行系统分解。流行的范型有：过程性的，逻辑的，面向存取的，面向进程的，面向对象的，函数型的，说明性的。每个范型都有它的支持者和用户，每个范型都特别适合于某种类型的问题或子问题。

注释：例如，逻辑程序设计范型是基于规则的，它把有关问题的知识分解成一组具体规则，用语言的“if\_then”等结构来表示这些规则。面向存取范型是一种在构造用户界面方面很有用的技术。

此外，每一个范型都用**不同的方式考虑问题**，每一个范型都使用**不同的方法来分解问题**，而且每一个范型都导致不同种类的块、过程、产生规则。下面主要讨论三种范型。研究的目的是帮助我们找到解决问题的入手点。

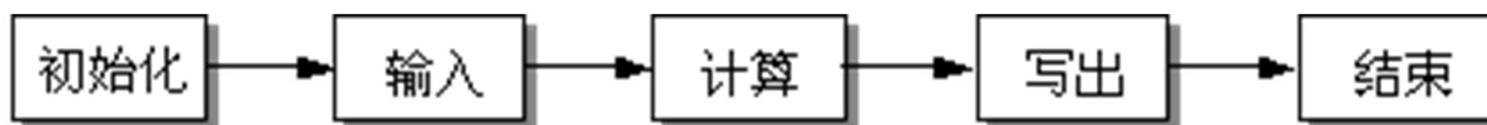
首先介绍面向过程范型，并将它与另外两个范型，即面向对象与面向进程的范型进行比较。考虑一个简单的**draw**程序，用户使用该程序可在屏幕上画一些简单的形状。**下图**显示了这样一个系统的概念设计，它的实现使用了事件驱动方法。



**draw (作图) 程序的概念模型**

# 面向过程范型

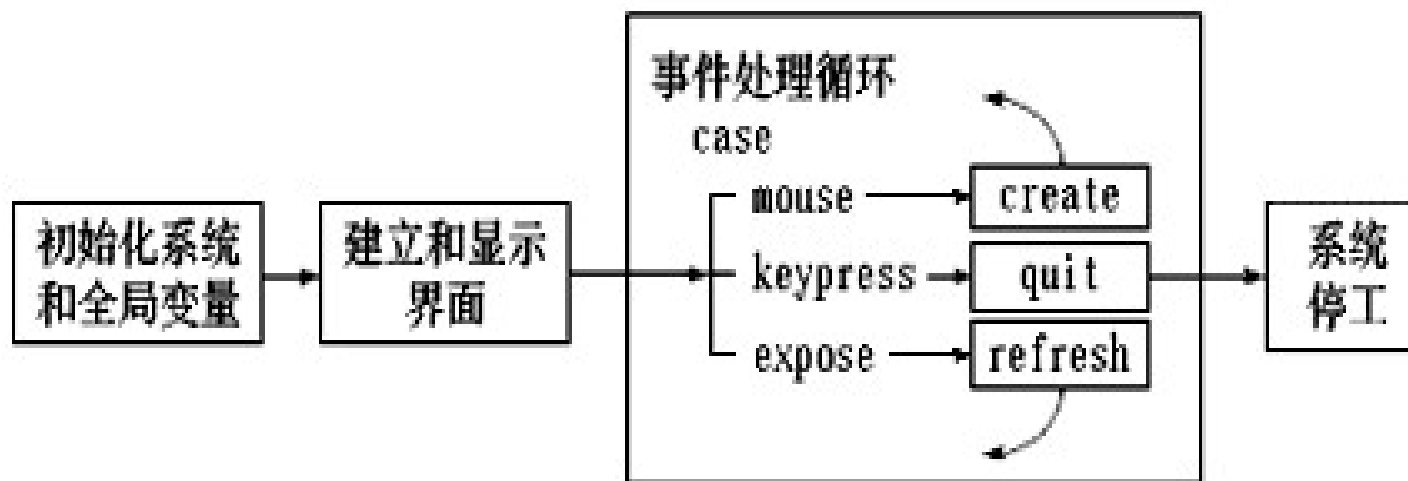
面向过程范型是使用最广泛、历史最长的软件范型。它产生过程的抽象，这些抽象把软件视为处理流，并定义成由一系列步骤构成的算法。每一步骤都是带有预定输入和特定输出的一个过程，把这些步骤串联在一起可产生合理的稳定的贯通于整个程序的控制流，最终产生一个简单的具有静态结构的体系结构，如下图所示。



面向过程的系统结构

面向过程范型侧重建立构成问题处理的流程。数据抽象、数据结构是根据算法步骤的要求开发的，它贯穿于过程，提供过程所要求操作的信息。系统的状态是一组全局变量，这组全局变量保存状态的值，把它们从一个过程传送到另一个过程。

下图给出了一种draw程序的面向过程的程序结构。在此例中，系统的各个任务顺序地排列，必要时也可以并发地安排。系统设计时，首先确定任务顺序，再定义支持过程基本操作的数据。

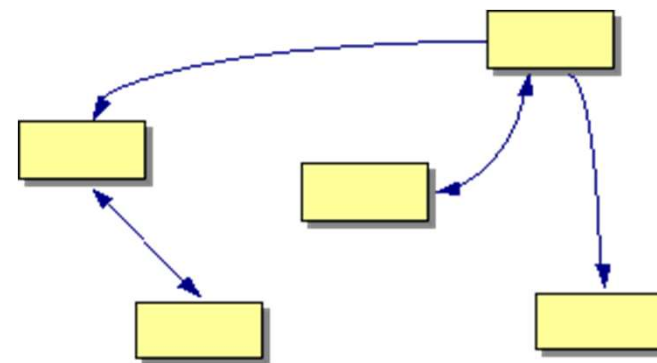


draw程序的过程性体系结构



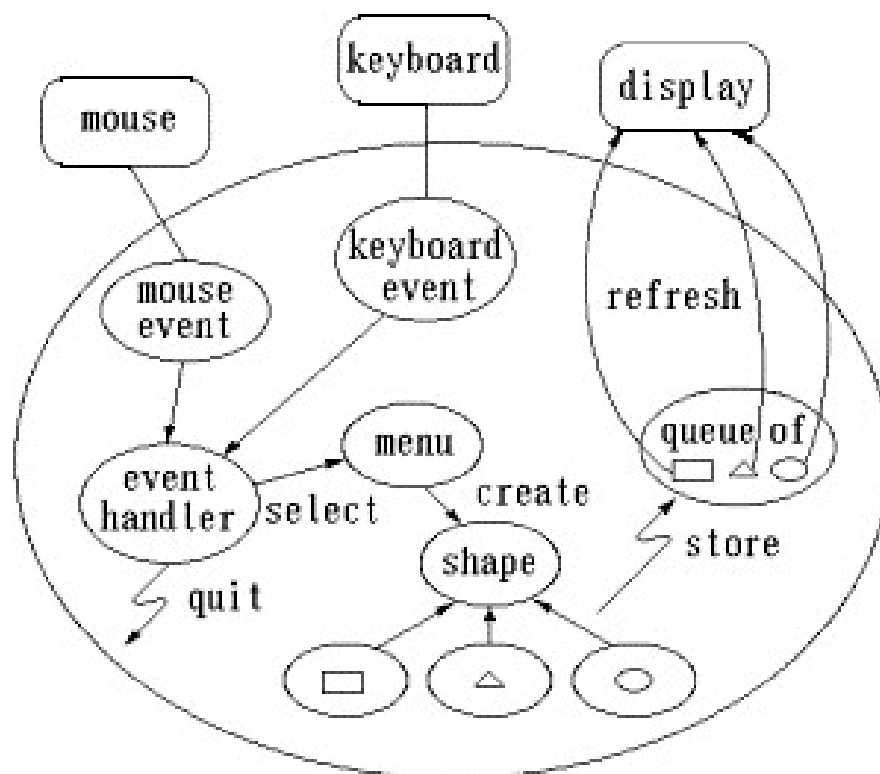
# 面向对象范型

面向对象系统中的对象是**数据抽象与过程抽象的综合**。系统的状态保存在各个数据抽象的核心所定义的数据存储中。控制流包含在各个数据抽象中的操作内。不像在过程性范型里那样，把数据从一个过程传送到另一个过程，而是把控制流从一个数据抽象通过消息传送到另一个数据抽象。完成的系统体系结构更复杂但也更灵活，如图所示。把控制流分离成块，这样可以把复杂的动作视为各个局部间的相互作用。



面向对象的系统结构

在面向过程的范型中优先考虑的是过程抽象，而在面向对象的范型中优先考虑的是实体，即问题论域的对象。在面向对象的范型中，把标识和模型化问题论域中的主要实体做为系统开发的起点，主要考虑对象的行为而不是必须执行的一系列动作。



## draw程序的面向对象 体系结构

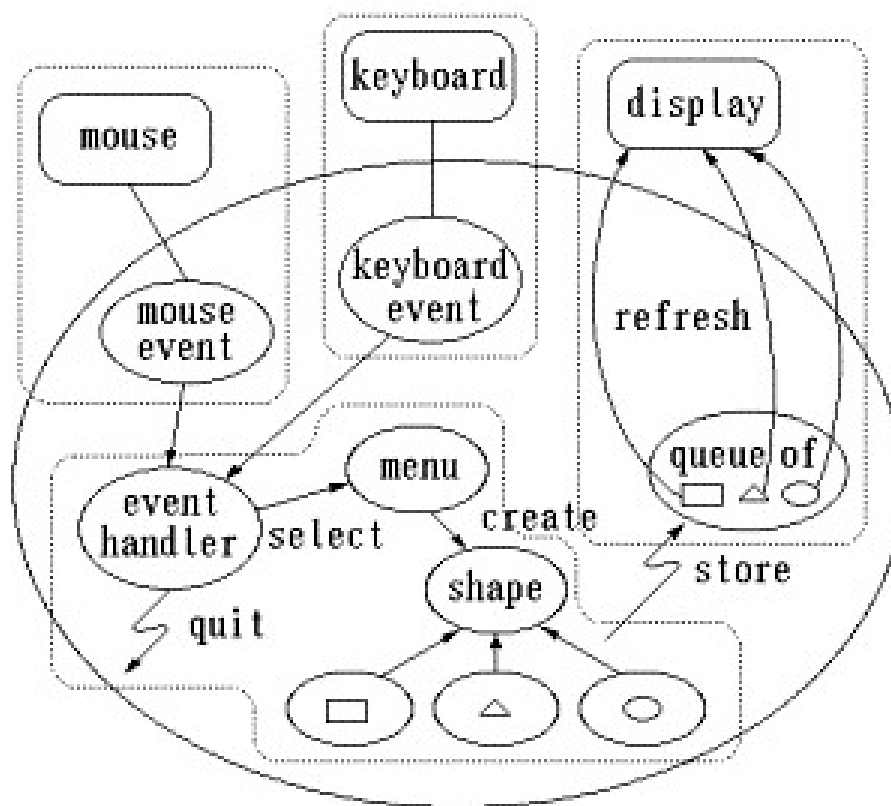
在面向过程的系统中，系统结构基于要执行的任务，改变其中一个成分可能需要改变其他所有的成分。而在面向对象的系统中，系统结构基于对象之间的交互，改变其中某一成分通常只具有局部影响。



# 面向进程的范型

面向进程的范型是把一个问题分解成**独立执行的模块**。让不只一个程序同时运行。这些进程互相配合，解决问题。面向进程范型产生的主要的块是进程。一个进程中的活动独立于其它进程的活动，但可以要求从其它进程得到信息，或为其它进程提供信息。甚至可以异步处理，仅需要进程暂停发送或接收信息。

在面向对象范型中，各个对象是相对独立的，但也存在单线索(单线程)控制。面向进程范型支持与面向对象范型相同的封装，但可提供多线索(多线程)执行。



对于draw程序的进程体系结构

面向进程范型的用户群体相对比较小；但这种范型将给用户提供很成熟的技术。许多主流语言，像Ada语言，通过tasking（派任务）功能支持这种范型。Ada还提供了一个抽象界面来产生进程，而C / UNIX环境则允许设计者直接存取操作系统级服务。

总之，面向进程范型给设计者提供了一种方法，可在操作系统环境下把问题分解成一组独立的实体，这些实体可以并行操作及以各种方式互相通信。面向进程范型在信息隐蔽与抽象方面非常类似于面向对象范型。



# 面向对象的概念

## 1、面向对象的思想

关于“面向对象”，有许多不同的看法。**Coad**和**Yourdon**给出了一个定义：“面向对象 = 对象 + 类 + 继承 + 消息通信”。如果一个软件系统是使用这样4个概念设计和实现的，则认为这个软件系统是面向对象的。一个面向对象的程序的每一成份应是对象，计算是通过新的象的建立和对象之间的消息通信来执行的。

## 2、对象(object)

一般意义来讲，对象是现实世界中存在的一个事物。可以是物理的，如一个家具或桌子。可以是概念上的，如一个开发项目。对象是构成现实世界的一个独立的单位，具有自己的静态特征（用属性描述）和动态特征（服务）。例如：人的静态特征：姓名、性别、年龄等，动态特征：衣、食、住、行等。

对象可以划分为以下几种类型：

- 外部实体**：与该系统交换信息的外部设备、相关子系统、操作员或用户等。
- 信息结构**：问题信息域中的概念实体，如信号、报表、显示信息等。
- 需要记忆的事件**：在系统运行过程中可能产生并需要系统记忆的事件，如单击鼠标左键、击打键盘"?"键等。
- 角色**：与软件系统交互的人员所扮演的角色，如经理、部长、技术支持等。
- 组织机构**：有关机构，如单位、小组等。
- 位置**：作为系统环境或问题上下文的场所、位置，如客户地址、收件人(机构)地址等。
- 操作规程**：如操作菜单、某种数据输入过程等。

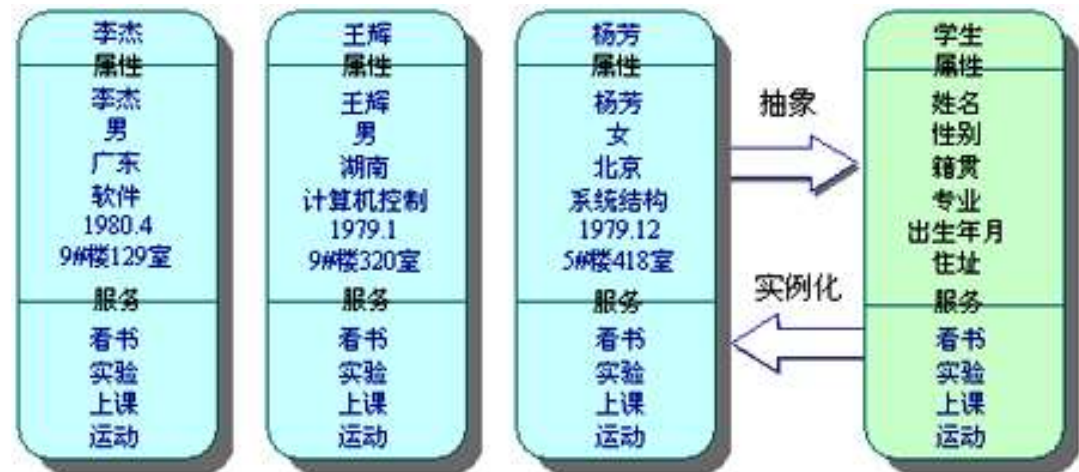
在标识对象时必需注意遵循“信息隐蔽”的原则：必需将对象的属性隐藏在对象的内部，使得从对象的外部看不到对象的信息是如何定义的，对象的状态通过给对象赋予具体的属性值而得到。它只能通过该对象的操作来改变。

对象实现了属性和服务的结合，使属性和服务封装于对象的统一体中。对象内的数据具有自己的操作，从而可灵活地专门描述对象的独特行为。具有较强的独立性；其内部状态不受或很少受外界的影响，具有很好的模块化特点。对象为软件复用打下了坚实的基础。



# 类（class）和实例（instance）

把具有相同特征和行为的对象归在一起就形成了类。类成为某些对象的模板，抽象地描述了属于该类的全部对象的属性和操作。属于某个类的对象叫做该类的实例。对象的状态则包含在它的实例变量，即实例的属性中。如图所示。从“李杰”、“王辉”和“杨芳”等对象可得到类“学生”，而这些对象就称为该类的实例。

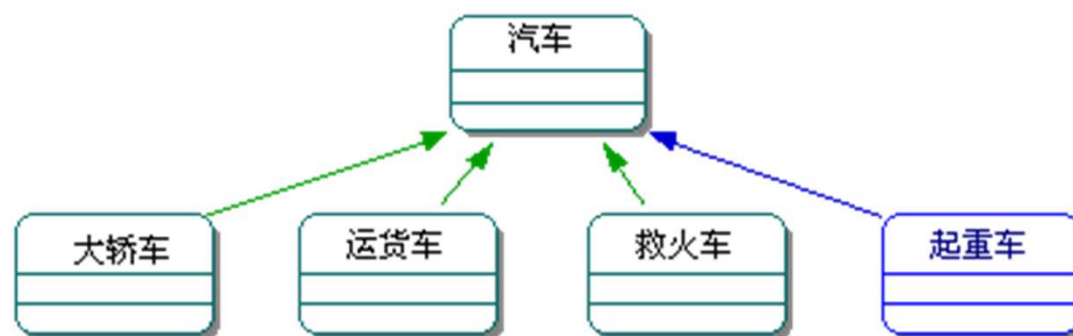


类定义了各个实例所共有的结构，类的每一个实例都可以使用类中定义的操作。实例的当前状态是由实例所执行的操作定义的。



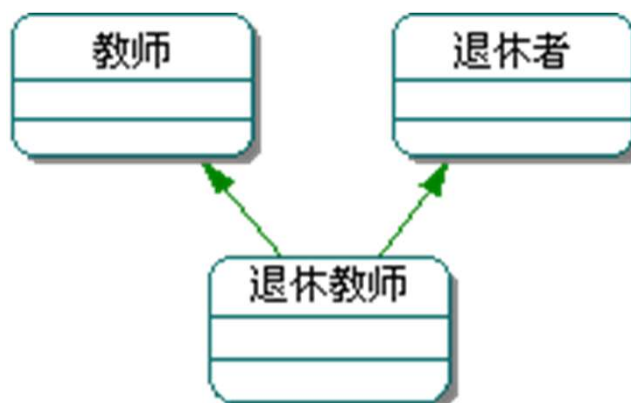
## 继承 (inheritance)

如果某几个类之间具有共性的东西（信息结构和行为），抽取出来放在一个一般类中，而将各个类的特有的东西放在特殊类中分别描述，则可建立起特殊类对一般类的继承。如下图所示。各个特殊类可以从一般类中继承共性，这样避免了重复。



特殊类对一般类的继承关系

如果一个类需要用到多个既存类的特征，可以从多个类中继承，称为多继承。例如退休教师是继承退休者和教师这两个类的某些特征或行为而得到的一个新类。如图6.11所示。





## 多态性和动态绑定

对象互相通信，即一个对象发消息给另一个对象，执行某些操作或又发消息给另外的对象，从而执行系统的功能。发送消息的对象可能不知道另一个对象的类型是什么。如在C程序中使用命令 `ClearInt()` 时要严格区分该命令适合一个整数，还是一个整数数组。但在C++情形，`ClearInt()` 对两者都适用，它自己判断对象是哪一个。这就是多态性。它意味着一个操作在不同类中可以有不同的实现方式。如清零操作 `ClearInt()` 针对消息对象是 `int array` 还是 `int`，其实现是不同的。

动态绑定把调用与目标模块的连接延迟到运行时进行。这样，只有发送消息时才与接收消息实例的一个操作绑定。它与多态性可以使我们建立的系统更灵活，易于扩充。



# 面向对象风格的开发过程

面向对象范型不仅是一些具体的软件体系结构风格，而且是一整套关于如何看待软件系统与现实世界的关系以及如何如何进行系统构造的软件方法学。用面向对象开发方法构造的软件具有以下特点：

- 面向对象的技术建立的模型与客观世界一致，因而便于理解；
- 适应变化的需要，修改局限在模块中；
- 可复用性。

面向对象软件的开发过程开始于问题论域，经历从问题提出到解决的一系列过程。下面具体说明在过程中的这些步骤。

## 1、分析阶段

分析阶段包括两个步骤：论域分析和应用分析。它们都要标识问题论域中的抽象。在分析中，需要找到特定对象，基于对象的公共特性把它们组合成集合，标识出对这个问题的一个抽象。同时要标识抽象之间的关系，并建立对象之间的消息连接。

在分析过程中标识的对象不是孤立的项，在给定的问题环境中，多数对象都是与其他的对象相互有影响的。这些相互影响就构成应用系统的结构中对象之间的联系。这些联系在应用系统中常常用对象之间的消息来表示，叫做消息连接。消息是一个对象发出的让另一个对象做某个动作的请求。在一个面向对象的应用中的控制流由两部分构成：每个单独的操作内部的控制流加上对象之间的消息模式。

### 1) 论域分析

论域分析开发问题论域模型。论域分析应当在应用分析之前进行，我们在了解问题之前应当对问题敞开思想考虑，考察问题论域内的一个较宽的范围，分析覆盖的范围应比直接要解决的问题更多。

论域分析最大的价值是可复用类的开发，这些类表示了一个问题论域中的基本概念，它们形成的软件库还可支持许多应用的开发。

面向对象软件的开发过程开始于问题论域，经历从问题提出到解决的一系列过程。下面具体说明在过程中的这些步骤。

## 1、分析阶段

分析阶段包括两个步骤：论域分析和应用分析。它们都要标识问题论域中的抽象。在分析中，需要找到特定对象，基于对象的公共特性把它们组合成集合，标识出对这个问题的一个抽象。同时要标识抽象之间的关系，并建立对象之间的消息连接。

在分析过程中标识的对象不是孤立的项，在给定的问题环境中，多数对象都是与其他的对象相互有影响的。这些相互影响就构成应用系统的结构中对象之间的联系。这些联系在应用系统中常常用对象之间的消息来表示，叫做消息连接。消息是一个对象发出的让另一个对象做某个动作的请求。在一个面向对象的应用中的控制流由两部分构成：每个单独的操作内部的控制流加上对象之间的消息模式。

## (2) 应用分析

应用(或系统)分析细化在论域分析阶段所开发出来的信息，并且把注意力集中于当前要解决的问题。因为通过论域分析，分析人员具有了较宽的论域知识，因而能开发出更好的抽象。 课文注释：传统的方法学人为地把系统分析与设计过程分离开来，这就把系统的设计和开发人员与对问题了解最清楚的专家隔离开来。而在面向对象方法中统分析与高层设计之间的差别正在逐渐变得模糊。





## 2、高层设计

在一个纯面向对象环境中，软件体系结构与类设计常常是同样的过程，但还是应当把体系结构与类的设计分开。在高层设计阶段，设计应用系统的顶层视图。这相当于开发一个代表系统的类，通过建立该类的一个实例并发送一个消息给它来完成系统的“执行”。

## 3、类的开发

根据高层设计所标识的对各个类的要求和类的规格说明进行类的开发。因为一个应用系统往往是一个类的继承层次。对这些类的开发是最基本的设计活动。

类的开发遵循类的生存期。它们的开发独立于所希望的应用，但支持这个应用。

## 4、实例的建立

建立各个对象的实例，实现问题的解决方案。

在分析阶段所标识的关系是应用级的关系。这些关系通过实例之间传送的消息来表示。在实现阶段则需要建立实例之间的通信通道。这些通道可以通过把引用从一个对象传递到另一个对象来建立。



# 面向对象分析（OOA）和模型化

面向对象分析强调直接对问题论域中的各种事物建立**OOA**模型中的对象，用对象的属性和服务分别描述对象的静态特征和动态行为。同时，在**OOA**模型中保存了问题论域中各个事物之间的关系，把具有相同属性和相同服务的对象归于一类，用一般化/特殊化关系（分类结构）和整体/部分关系（组装结构）描述事物间的结构关系，用实例连接和消息连接描述事物之间的静态联系和动态联系，从而忠实地反映了问题论域的原貌。

建立**OOA**模型，要运用如下的**5**个基本原则：

- 1、建立信息域模型；
- 2、描述功能；
- 3、表达行为；
- 4、划分功能、数据、行为模型，揭示更多的细节；
- 5、用早期的模型描述问题的实质，用后期的模型给出实现的细节。

这些原则形成**OOA**的基础。



# 面向对象分析（OOA）和模型化

**OOA**的目的是定义所有与待解决问题相关的类(包括类的操作和属性、类与类之间的关系以及它们表现出的行为)。为此，**OOA**需完成的任务是：

- (1) 软件工程师和用户必须充分沟通，以了解基本的用户需求；
- (2) 必须标识类(即定义其属性和操作)；
- (3) 必须定义类的层次；
- (4) 应当表达对象与对象之间的关系(即对象的连接)；
- (5) 必须模型化对象的行为；
- (6) 反复地做任务(1)—(5)，直到模型建成。

OOA和OOD之间有密切的衔接关系，从OOA到OOD是一个逐渐扩充模型的过程。分析处理以问题为中心，可以不考虑任何与特定计算机有关的问题，而OOD则把我们带进了面向计算机的“实地”开发活动中去。通常，OOD分为两个阶段，即高层设计和低层设计。高层设计建立应用的体系结构。低层设计集中于类的详细设计。

可以使用以下几个简单规则来开发一个应用的体系结构：

## (1) 最小化各部件间的通信

在子系统的各个高层部件之间的通信量应当达到最小。一个用户界面应当能够自行处理交互、错误改正和硬件控制，而不需打扰主应用。仅当通过界面输入一个请求的时候，才需要把它转送到应用的其余部分。

## (2) 隐藏复杂性

子系统应当把那些成组的类打包，形成高度的内聚。

## (3) 逻辑功能分组

虽然输入和输出设备可能相互间不通信，但逻辑上把它们归组到一个处理输入 / 输出的子系统中。这样比较容易识别并定位问题论域中的事件。

类与通过概念封装的子系统十分类似。事实上，每个子系统都可以被当做一个类来实现，这个类聚集它的构件，提供了一组操作。



# OOA/OOD方法概述

## 1、Booch方法

Booch方法包含“微开发过程”和“宏开发过程”。微开发过程定义了一组任务，并在宏开发过程的每一步骤中反复使用它们，以维持演进途径。

Booch的OOA宏开发过程概述如下：

- (1) 标识类和对象 提出候选对象，进行行为分析，标识相应的场景，为每个类定义属性和操作；
- (2) 标识类和对象的语义 选择场景并进行分析，识别为实现期望的行为应有的责任，为均衡各种行为划分责任，对选定的每一个对象列举它们应承担的角色和行为，为履行责任定义各种操作，寻找对象之间的协作关系；
- (3) 定义类与对象间的关系 定义对象间存在的依赖关系，描述每一个参与对象的角色，通过走查场景进行确认；
- (4) 进行一系列求精 对上述的工作建立相应的图示，定义适当的类层次，基于类的共性建立类间结构；
- (5) 实现类和对象 在OOA的情形下完成分析模型。

微开发过程定义了宏开发过程中每一步可用的一组设计任务，因此可以保证开发进程的不断进展。

**Booch**的**OOD**微开发过程可概述如下：

**(1) 规划系统的体系结构**

将概念近似、关系密切的对象聚合到各个分离的体系结构区域内，根据抽象层次将各对象分层，标识相关的场景，创建设计原型，基于设计原型利用使用场景走查、籍此来检查设计原型；

**(2) 确立设计方针**

定义特定论域的方针（包括管理各种操作和属性如何使用的规则），为存储管理、错误处理和其他基本管理功能定义论域专用的方针，建立描述这些方针语义的场景，为每一个方针创建原型，完善和细化原型，评审每一个方针确保方针体现了体系结构的设想；

**(3) 部署规划**

根据优先级组织在**OOA**中已建立的场景，将对应的体系结构方案分配给场景，渐增式地设计和构造每一个体系结构方案，根据需要调整渐增设计的目标和进度。



## 2、Rumbaugh方法

Rumbaugh和他的同事提出的对象模型化技术(OMT)用于分析、系统设计和对象级设计。分析活动建立三个模型：对象模型(描述对象、类、层次和关系)，动态模型(描述对象和系统的行为)，功能模型(类似于高层的DFD，描述穿越系统的信息流)。

Rumbaugh的OOA过程概述如下：

- (1) 建立对问题范围的陈述；
- (2) 建立对象模型标识与问题相关的类，定义各对象的属性和对象间的关联，定义对象间的连接，定义对象间的继承关系；
- (3) 开发动态模型准备问题解决的场景，定义场景中的各种事件并为每一个场景建立事件追踪图，构造事件流图示，开发各个类的状态图示，评审行为的一致性和完整性；
- (4) 构造系统的功能模型标识系统的输入和输出，利用数据流图表达流的变换，为每一个功能开发加工规格说明，加入各种约束和优化标准。

### 3、Coad和Yourdon方法

Coad和Yourdon方法常常被认为是最容易学习的OOA方法。建模符号相当简单，而且开发分析模型的导引直接明了。其OOA过程概述如下：

- (1) 使用"要找什么"准则标识对象；
- (2) 定义对象之间的一般化/特殊化结构；
- (3) 定义对象之间的整体/部分结构；
- (4) 标识主题(系统构件的表示)；
- (5) 定义属性及对象之间的实例连接；
- (6) 定义服务及对象之间的消息连接。

Coad和Yourdon的OOD方法不但强调问题的解决，还强调系统的基本管理，其OOD过程概述如下所示。

- (1) **问题论域部分**将所有特定论域类组合到应用中来，为应用中的类设计适当的类层次，必要时简化继承，细化设计以改进性能，靠方法与数据管理部分的接口，必要时细化并加入低层的对象，评审设计结果并审查对分析模型做了哪些增补。
- (2) **人机交互部分**定义用户（参与者）类型，建立任务场景，设计用户命令层次，细化用户交互序列，设计相关的类和类层次，适当地集成GUI（图形用户交互）类。
- (3) **任务管理部分**标识任务类型（如事件驱动、时钟驱动），建立任务的优先级，标识协调其他任务的协调者任务，为每一个任务设计合适的对象。
- (4) **数据管理部分**设计数据结构及其布局，设计管理数据结构所需的服务，标识可以协助实现数据管理的工具，设计适当的类和类层次。



#### 4、Jacobson方法

也称为OOSE(面向对象软件工程)。Jacobson方法与其它方法的不同之处在于他特别强调利用使用用例(use case)来描述用户与系统之间如何交互的场景。

Jacobson方法概述如下：

- (1) 标识系统的用户和它们的整体责任；
- (2) 建立需求模型定义参与者及其职责，为每一个参与者标识使用实例，准备系统对象和关系的初步视图，应用使用实例作为场景评审需求模型以确定模型的有效性；
- (3) 建立分析模型利用与参与者交互的信息来标识界面对象，创建界面对象的结构视图，表达对象的行为，分离出每个对象的子系统和模型，应用使用实例作为场景评审分析模型以确定模型的有效性。

由Jacobson推出的OOSE设计模型强调对OOSE分析模型的可追踪性，其OOD过程可概述如下：

- (1) 为使分析模型适合现实世界环境的要求，对该模型进行适应性修改；
- (2) 定义块（block）作为主要的设计对象块是设计抽象，可能代表某个对象集合。在这一步定义可实现相关分析对象的块，标识界面块、实体块和控制块，描述在执行过程中块之间如何通信，标识在块之间传送的激活信号（token）以及它们之间的通信顺序；
- (3) 创建交互图来描述激活信号如何在块间传送；
- (4) 将块组合为子系统；
- (5) 评审设计工作。

## 5、统一的OOA方法(UML)

统一的建模语言(UML)已经在企业中广泛使用, 它把Booch、Rumbaugh和Jacobson等各自独立的OOA和OOD方法中最优秀的特色组合成一个统一的方法。UML允许软件工程师使用由一组语法的语义的实用的规则支配的符号来表示分析模型。

在UML中用5种不同的视图来表示一个系统, 这些视图从不同的侧面描述系统。每一个视图由一组图形来定义。这些视图概述如下:

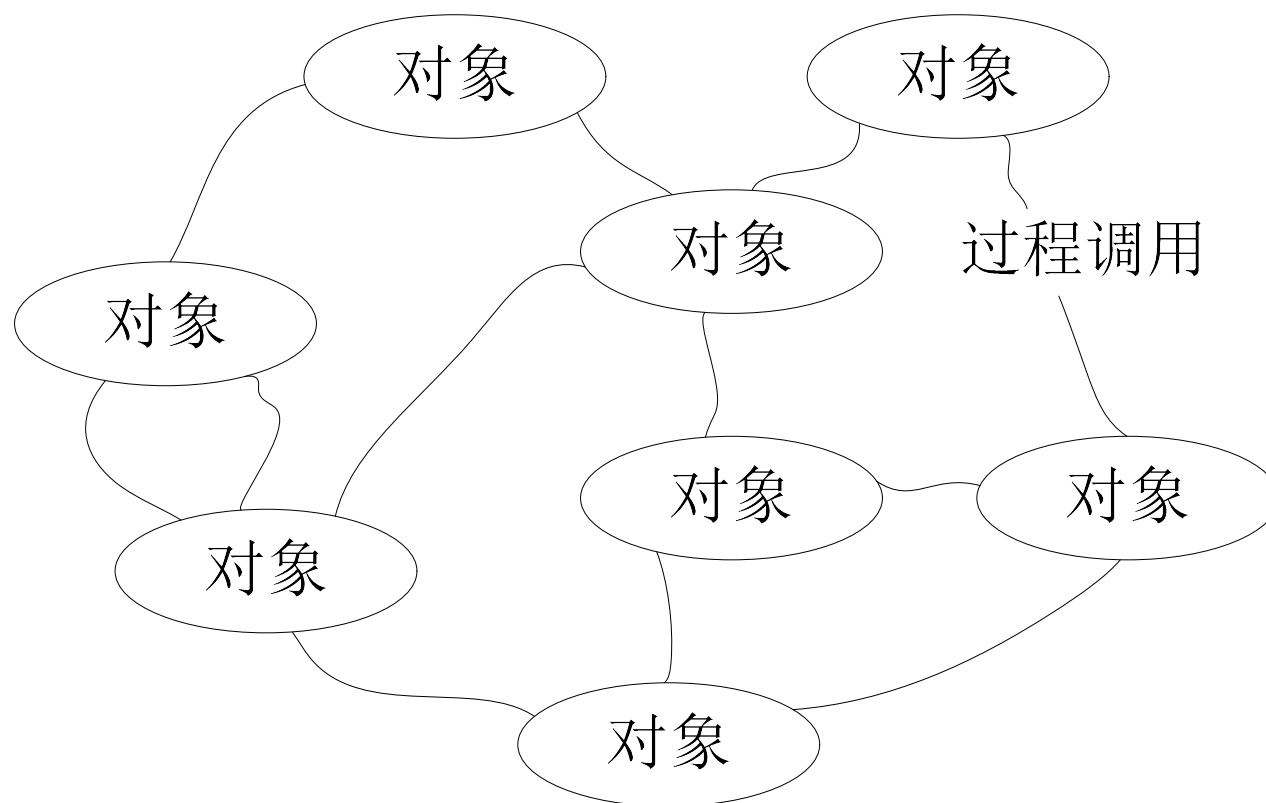
- (1) **用户模型视图**这个视图的主要目的是获取用户对系统的要求, 它从用户(在UML中叫做参与者)角度来表示系统, 并利用使用用例(use case)建立需求模型, 用以描述来自终端用户方面的可用的场景。
- (2) **结构模型视图**这个视图使用基本的需求作为导引来选择类和对象, 识别系统内部的数据和功能性, 对静态结构(类、对象和关系)模型化。
- (3) **行为模型视图**这种视图表示了系统动态和行为。它还描述了在用户模型视图和结构模型视图中所描述的各种结构元素之间的交互和协作。
- (4) **实现模型视图**将系统的结构和行为表达成为易于转换为实现的方式。
- (5) **环境模型视图**表示系统实现环境的结构和行为。

# 面向对象设计OOD

尽管上面所列举的OOD方法的术语和过程步骤各不相同，但整体上OOD过程基本上是一致的。为完成面向对象设计，需要做的工作有以下四个：

- (1) 以可实现的方式描述每一个子系统为每一个子系统分配处理器和任务，选择实现数据管理、界面支持和任务管理的设计策略，为系统设计合适的控制机制，评审并进行权衡；
- (2) 对象设计设计每一个操作的程序流程，定义所需的内部类，设计所需的内部数据结构；
- (3) 消息设计使用对象之间的协作和对象-关系模型设计消息模型；
- (4) 评审设计模型并在必要时进行迭代。我们讨论的设计步骤是递进的，它们可以渐增地执行，伴随着附加的OOD活动，直到完成整个设计。

- 面向对象组织结构有两个重要特点：
  - (1)对象负责维护其表示的完整性
  - (2)对象的表示对其他对象而言是隐蔽的。





- 优点

- （1）对象隐藏了其实现细节，所以可以在不影响其它对象的情况下改变对象的实现，不仅使得对象的使用变得简单、方便，而且具有很高的安全性和可靠性。
- （2）设计者可将一些数据存取操作的问题分解成一些交互的代理程序的集合。

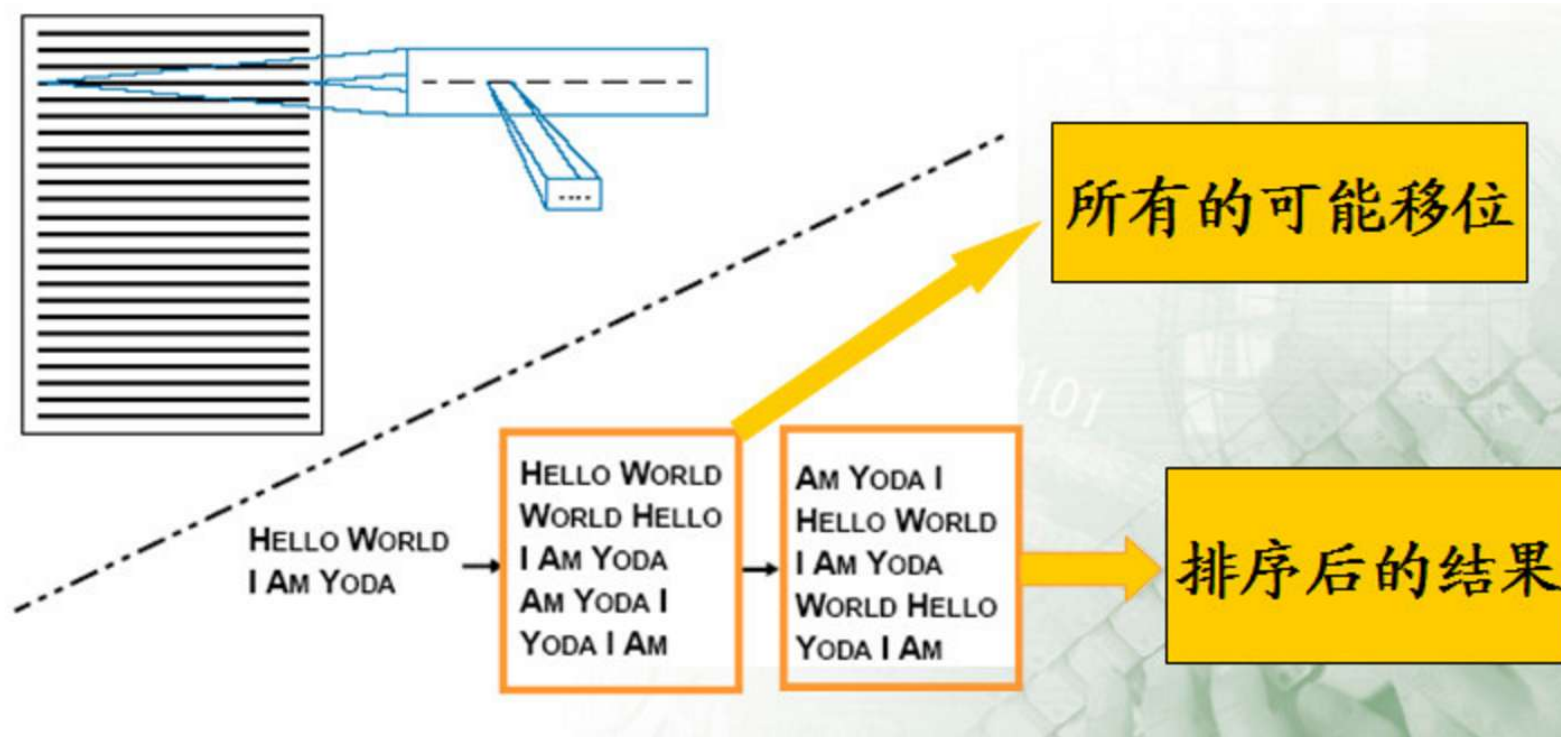
- 缺点

- 当一个对象和其它对象通过过程调用等方式进行交互时，必须知道其它对象的标识。无论何时改变对象的标识，都必须修改所有显式调用它的其它对象，并消除由此带来的一些副作用。

# KWIC问题



- KWIC(Key Word In Context), Parnas (1972)
- KWIC索引系统接受一些行，每行有若干字，每个字由若干字符组成；每行都可以循环移位，亦即重复地把第一个字删除，然后接到行末；KWIC把所有行的各种移位情况按照字母表顺序输出







A third way for styles to  
be combined is to  
elaborate one level of

Input(输入)



A third way for styles to  
third way for styles to A  
way for styles to A third  
for styles to A third way  
styles to A third way for  
to A third way for styles

be combined is to  
combined is to be  
is to be combined  
to be combined is

elaborate one level of  
one level of elaborate  
level of elaborate one  
of elaborate one level

Circular shifts(循环移位)



A third way for styles to  
be combined is to  
combined is to be  
elaborate one level of  
for styles to A third way  
is to be combined

level of elaborate one  
of elaborate one level  
one level of elaborate  
styles to A third way for  
third way for styles to A  
to A third way for styles  
to be combined is  
way for styles to A third

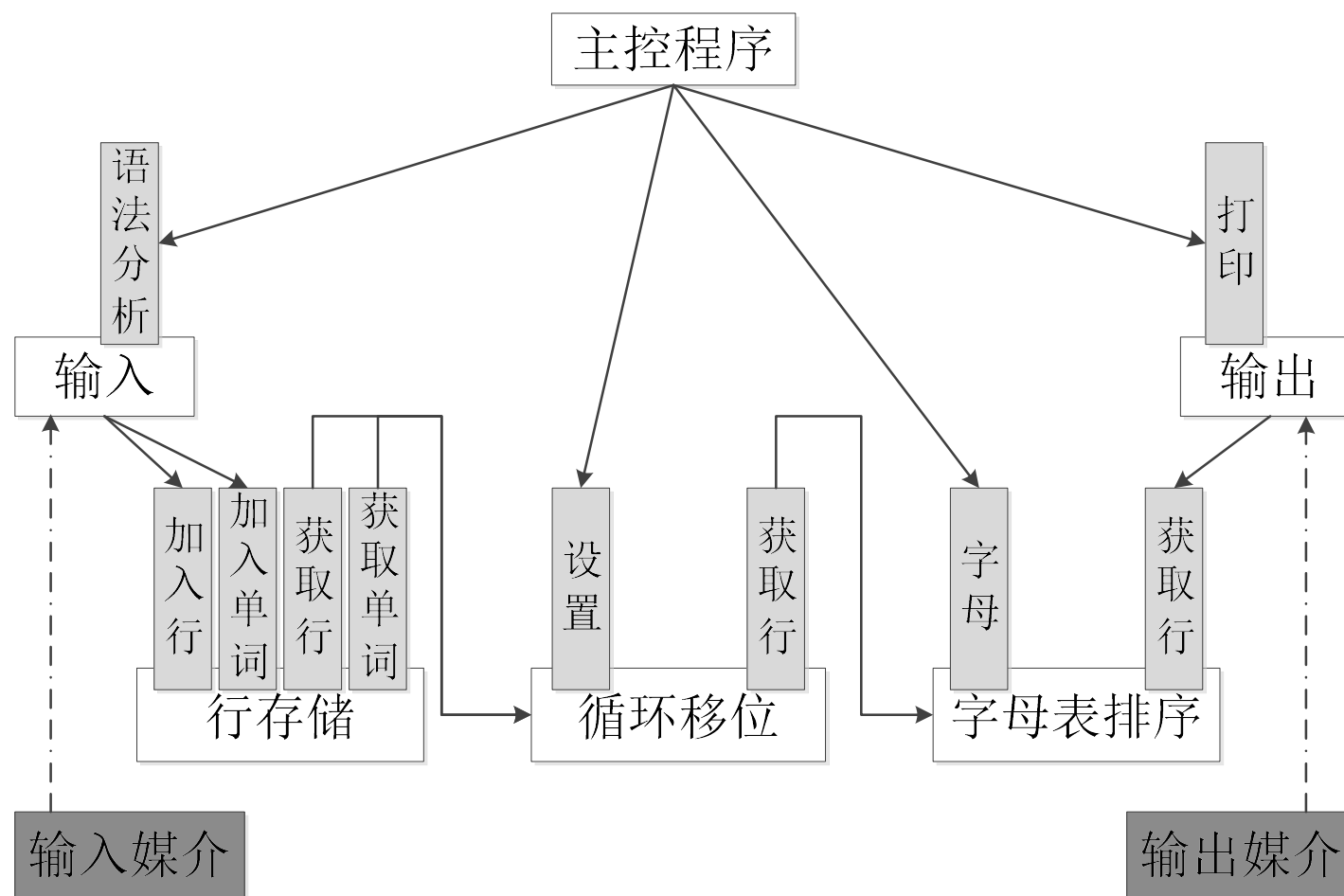
Output(输出)

# 面向对象风格



南京理工大学  
NANJING UNIVERSITY OF SCIENCE & TECHNOLOGY

- 应用实例：KWIC（Key Word in Context）检索系统



图例： □ 模块（对象）    ■ 公共接口    ■ 输入/输出媒介  
——→ 方法调用    - - - -> 系统输入/输出

# 事件风格？

